Lab sheet 10

1.Objectives

On completion of this lab, students will be able to:

1. **Design and implement a secure RESTful API with JWT-based authentication and role-based access control (RBAC) for at least two user roles (Admin, User).**

   Ans: To implement secure user authentication using JWT and role-based access control (RBAC).

2. **Integrate real-time bidirectional communication using WebSockets (Socket.io) to push live notifications or chat updates between connected clients.**

   Ans: To implement real-time notifications using Socket.io.

3. **Implement server-side caching with Redis to reduce database load and measure the performance improvement using response-time benchmarking.**

   Ans: To use Redis caching to improve API response time.

4. **Apply industry-standard secure coding practices — password hashing (bcrypt), request validation, rate limiting, and CORS policy configuration.**

   Ans: To develop a responsive React frontend with global state management.

5. **Build a responsive React frontend with global state management (Redux Toolkit or Context API + useReducer) that consumes the protected APIs and reflects real-time updates.**

   Ans: To implement security features such as bcrypt, request validation, rate limiting, and Helmet.

6. **Containerize the entire application (frontend, backend, MongoDB, Redis) using Docker and orchestrate it with Docker Compose.**

   Ans: To test the application using Jest, Supertest, and React Testing Library.

7. **Write and execute automated tests (unit tests for backend controllers using Jest/Supertest, and component tests for React using React Testing Library).**

   Ans: To containerize and deploy the complete application using Docker Compose.

2. Pre-requisites

Area	Expected Knowledge
Backend	Node.js, Express routing/middleware, MongoDB & Mongoose schemas
Frontend	React functional components, hooks, React Router
Auth	Basic understanding of JWT, HTTP headers, cookies
Tools	Postman/Insomnia, Git, npm/yarn, basic Docker CLI

Ans:

Area	Required Knowledge
Backend	Node.js, Express.js, REST APIs
Database	MongoDB,Mongoose
Frontend	React.js, Hooks, React Router
Authentication	JWT, cookies, bcrypt
Real-time communication	Socket.io
Caching	Redis
Testing	Jest, Supertest, React Testing Library
Tools	VS Code, Postman, Git, Docker

3. Problam statement

You are required to build a "CampusConnect" application — a role-based event and announcement portal for a college. It must support two roles: ADMIN (creates/edits/deletes events and announcements) and STUDENT (views events, RSVPs, and receives live notifications when a new announcement is posted). The system must be secure, performant under repeated reads, and fully containerized for deployment.

Ans: CampusConnect is a secure college event and announcement portal with two roles: ADMIN and STUDENT.

- Admin: Creates, updates, and deletes events and announcements.
- Student: Views events, searches events, RSVPs, and receives real-time notifications.
- System: Uses JWT authentication, Redis caching, automated testing, and Docker containerization.

4.Tasks

Task 1 — Authentication & RBAC

- **Create User schema with fields: name, email, passwordHash, role (ADMIN/STUDENT).**

- **Hash passwords using bcrypt before saving (min. 10 salt rounds).**
- **Implement /api/auth/register and /api/auth/login issuing a signed JWT (access token, 15 min expiry) and a refresh token (httpOnly cookie, 7 days).**
- **Write custom middleware authenticate and authorize(role) to protect routes — Admin-only routes must reject Students with 403.**
- **Implement token refresh endpoint /api/auth/refresh without requiring re-login.**

Ans: Create a User schema containing name, email, passwordHash, and role.
Install the required packages:

npm install express mongoose bcrypt jsonwebtoken cookie-parser dotenv
npm install express-rate-limit zod helmet cors

Example user model:

```
const mongoose = require("mongoose");
const userSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: {
    type: String,
    required: true,
    unique: true,
    lowercase: true
  },
  passwordHash: { type: String, required: true },
  role: {
    type: String,
    enum: ["ADMIN", "STUDENT"],
    default: "STUDENT"
  }
});
module.exports = mongoose.model("User", userSchema);
```

Example password hasaing:

```
const bcrypt = require("bcrypt");
```

Example JWT generation:

```
const passwordHash = await bcrypt.hash(password, 10);
const jwt = require("jsonwebtoken");
const accessToken = jwt.sign(
  { id: user._id, role: user.role },
```

```
  process.env.JWT_SECRET,
 { expiresIn: "15m" }
);
```

The refresh token should be stored in an httpOnly cookie with a seven-day expiry. Implement refresh-token rotation and server-side revocation. Never allow a public registration request to assign itself the ADMIN role.

Authentication middleware:

```
function authenticate(req, res, next) {
 const token = req.headers.authorization?.split(" ")[1];

 if (!token)
  return res.status(401).json({ message: "Token required" });

 try {
  req.user = jwt.verify(token, process.env.JWT_SECRET);
  next();
 } catch {
  return res.status(401).json({ message: "Invalid token" });
 }
}
```

Role authorization middleware:

```
const authorize = (role) => (req, res, next) => {
 if (req.user.role !== role) {
  return res.status(403).json({ message: "Access denied" });
 }
 next();
};
```

Example protected route:

```
app.post(
 "/api/events",
 authenticate,
 authorize("ADMIN"),
 createEvent
);
```

Required endpoints:

- POST /api/auth/register

- POST /api/auth/login
- POST /api/auth/refresh
- GET /api/events
- POST /api/events
- PUT /api/events/:id
- DELETE /api/events/:id
- POST /api/events/:id/rsvp

Task 2 — Real-Time Notifications [HARD]

- **Set up a Socket.io server alongside Express, authenticating socket connections using the JWT.**
- **When an Admin creates a new announcement, emit a new-announcement event to all connected Student sockets instantly.**
- **On the frontend, show a live toast/badge counter without requiring a page refresh.**
- **Handle reconnection gracefully if a client's socket drops.**

Ans: Install Socket.io:

npm install socket.io

Create a Socket.io server alongside Express and authenticate clients using their access token.

Backend implementation:

```
const { Server } = require("socket.io");
const jwt = require("jsonwebtoken");
const io = new Server(server, {
  cors: { origin: process.env.FRONTEND_URL }
});
io.use((socket, next) => {
  try {
    const token = socket.handshake.auth.token;
    socket.user = jwt.verify(token, process.env.JWT_SECRET);
    next();
  } catch {
    next(new Error("Authentication failed"));
  }
});
io.on("connection", (socket) => {
  if (socket.user.role === "STUDENT") {
    socket.join("students");
  }
});
```

After an Admin successfully creates an announcement, broadcast a notification:

```
io.to("students").emit("new-announcement", {
  title: announcement.title,
  message: announcement.message
});
```

Frontend implementation:

```
import { io } from "socket.io-client";
const socket = io(import.meta.env.VITE_API_URL, {
  auth: { token: accessToken },
  reconnection: true
});
socket.on("new-announcement", (data) => {
  console.log("New announcement:", data.title);
  // Update notification badge and show a toast.
});
```

Expected result: Students receive notifications immediately without refreshing the page. The frontend should update the notification counter and reconnect when the network connection is restored.

Task 3 — Redis Caching Layer

- **Install and configure Redis; cache the response of GET /api/events (list of events) with a 60-second TTL.**
- **Invalidate the cache automatically whenever an Admin creates, updates, or deletes an event.**
- **Benchmark and report average response time for cached vs. uncached requests (use autocannon or Postman runner, minimum 100 requests).**

Ans: Install Redis:

```
npm install redis
```

Redis connection:

```
const { createClient } = require("redis");
const redis = createClient({
  url: process.env.REDIS_URL
});
redis.on("error", (err) => console.error("Redis error:", err));
```

Connect to Redis before starting the backend:

```
await redis.connect();
```

Caching GET /api/events:

```
app.get("/api/events", async (req, res, next) => {
```

```
  try {
    const cacheKey = "events:list";
    const cached = await redis.get(cacheKey);
    if (cached) {
      return res.json({
        source: "cache",
        data: JSON.parse(cached)
      });
    }
    const events = await Event.find().lean();
    await redis.setEx(
      cacheKey,
      60,
      JSON.stringify(events)
    );
    res.json({ source: "database", data: events });
  } catch (error) {
    next(error);
  }
});
```

After creating, updating, or deleting an event, invalidate the cache:
`await redis.del("events:list");`

In a complete implementation, the cache key must also account for pagination, search terms, filters, and other query parameters. Otherwise, different requests could incorrectly share cached results.

Performance benchmarking:

Run the backend locally and use Autocannon to compare database and cache response times.
`npm install -g autocannon`

Run at least 100 requests for each test:
`npx autocannon -c 10 -a 100 http://localhost:5000/api/events`

First test after clearing the cache, then warm the cache and repeat. For a fair comparison, record the actual cache-hit rate and keep the dataset and request parameters identical.

Record the results:

Metric	Uncached	Cached
Average response time	___ ms	___ ms
Requests completed	___	___

Cache-hit rate	0%	___%

Calculate performance improvement using:

$$\text{Improvement (\%)} = \frac{T_u - T_c}{T_u} \times 100$$

Where Tu is uncached average latency and Tc is cached average latency.

Note: Fill the table with measured results; do not use assumed values as experimental findings.

Task 4 — Frontend with State Management

- **Build Login/Register, Dashboard (role-aware), Event List with pagination + search-by-title, and RSVP functionality.**
- **Use Redux Toolkit (or Context + useReducer) to manage auth state and event state globally.**
- **Persist the access token securely (in-memory store preferred over localStorage) and auto-attach it via an Axios interceptor.**
- **Protect frontend routes using a PrivateRoute/RoleRoute wrapper component.**

Ans: Create the React application using Vite:

```
npm create vite@latest frontend -- --template react
cd frontend
npm install
npm install axios react-router-dom @reduxjs/toolkit react-redux socket.io-client
```

Required frontend pages:

1. Login/Register: User authentication.
2. Dashboard: Different options for Admin and Student.
3. Event List: Displays events with pagination and title search.
4. Event Details: Shows event information and RSVP options.
5. Announcements: Displays notifications in real time.
6.

Redux Toolkit example:

```
import { createSlice } from "@reduxjs/toolkit";
const eventSlice = createSlice({
 name: "events",
 initialState: { items: [], loading: false },
 reducers: {
  setEvents: (state, action) => {
   state.items = action.payload;
  }
 }
```

```
});

export const { setEvents } = eventSlice.actions;
export default eventSlice.reducer;
```

Axios interceptor:

```
import axios from "axios";
const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL
});
api.interceptors.request.use((config) => {
  const token = authTokenStore.getAccessToken();
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});
export default api;
```

Here, authTokenStore represents an in-memory token store that must be implemented. Do not persist the access token in localStorage. Configure Axios with credentials when using the refresh-token cookie, and implement automatic refresh and retry carefully.

Route protection example:

```
import { Navigate } from "react-router-dom";
function RoleRoute({ user, allowedRole, children }) {
  if (!user) return <Navigate to="/login" />;
  if (user.role !== allowedRole) {
    return <Navigate to="/dashboard" />;
  }
  return children;
}
```

Frontend route protection improves navigation and user experience, but backend authorization remains essential for security.

Task 5 — Security Hardening

- **Add express-rate-limit on the login route (max 5 attempts / 15 min per IP).**
- **Validate all incoming request bodies using zod or express-validator; return structured 400 errors.**
- **Configure CORS to allow only the deployed frontend origin.**
- **Add security headers using helmet.**

Ans: Use Helmet, rate limiting, CORS, and Zod validation.

```
const helmet = require("helmet");
const cors = require("cors");
const rateLimit = require("express-rate-limit");
app.use(helmet());
app.use(cors({
  origin: process.env.FRONTEND_URL,
  credentials: true
}));
app.use(express.json());
const loginLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 5,
  standardHeaders: true,
  legacyHeaders: false
});
app.use("/api/auth/login", loginLimiter);
```

Request validation using Zod:

```
const { z } = require("zod");
const loginSchema = z.object({
  email: z.string().email(),
  password: z.string().min(8)
});
const result = loginSchema.safeParse(req.body);
if (!result.success) {
  return res.status(400).json({
    message: "Validation failed",
    errors: result.error.issues
  });
}
```

Additional security requirements:

- Keep secrets in environment variables.
- Use HTTPS in production.
- Validate and sanitize incoming data.
- Restrict Admin operations on the backend.
- Store refresh tokens securely and support revocation.
- Never expose password hashes or sensitive tokens in API responses.
- Use appropriate cookie settings such as httpOnly, secure in production, and a suitable sameSite policy.

Task 6 — Testing

- **Write at least 5 backend unit/integration tests (Jest + Supertest) covering: registration, login failure on wrong password, protected route rejection without token, admin-only route rejection for student role, and event creation success.**
- **Write at least 3 React Testing Library tests for the Login form and Event List rendering.**

Ans: Install testing packages in the backend:

npm install --save-dev jest supertest

Add this script to package.json:

```
{
  "scripts": {
    "test": "jest --runInBand"
  }
}
```

Backend test cases

Test	Expected result
1. Register a new user	User created successfully
2. Login with wrong password	401 Unauthorized
3. Access protected route without token	401 Unauthorized
4. Student accesses Admin-only route	403 Forbidden
5. Admin creates an event	201 Created

Example Supertest test:

```
const request = require("supertest");
const app = require("../app");
describe("Authentication API", () => {
  test("rejects login with wrong password", async () => {
    const response = await request(app)
      .post("/api/auth/login")
      .send({
        email: "student@example.com",
        password: "WrongPassword123"
      });
    expect(response.status).toBe(401);
  });
});
```

The test requires a properly configured test database and an existing test user.

React Testing Library:

Install frontend testing packages:

```
npm install --save-dev vitest jsdom \
  @testing-library/react \
  @testing-library/jest-dom \
  @testing-library/user-event
```

Write at least three frontend tests:

1. Login form renders email and password fields.
2. Login form displays a validation error for invalid input.
3. Event List renders event titles returned by the API.

Example test:

```
import { render, screen } from "@testing-library/react";
import { describe, it, expect } from "vitest";
import "@testing-library/jest-dom";
function EventList() {
  return <h2>Annual College Fest</h2>;
}
describe("Event List", () => {
  it("renders an event title", () => {
    render(<EventList />);
    expect(
      screen.getByText("Annual College Fest")
    ).toBeInTheDocument();
  });
});
```

This is a minimal component test. In the actual application, test the real Event List component and its loading, empty, and error states.

Task 7 — Containerization & Deployment

- **Write a Dockerfile each for the backend and frontend (multi-stage build for React).**
- **Write a docker-compose.yml orchestrating: frontend, backend, MongoDB, and Redis containers with proper environment variables and volume persistence for MongoDB.**
- **Verify the entire stack starts with a single docker compose up command and services can communicate via the Compose network.**

Ans:

Create the following project structure:

```
CampusConnect/
├── backend/
│   ├── src/
│   │   ├── models/
│   │   ├── routes/
│   │   ├── controllers/
│   │   ├── middleware/
│   │   └── app.js
│   ├── Dockerfile
│   └── package.json
├── frontend/
│   ├── src/
│   ├── Dockerfile
│   ├── nginx.conf
│   └── package.json
└── docker-compose.yml
```

Backend Dockerfile

```
FROM node:22-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
EXPOSE 5000
CMD ["node", "src/server.js"]
```

Frontend Dockerfile — Multi-stage build

```
FROM node:22-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build
FROM nginx:alpine
COPY --from=build /app/dist /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Docker Compose configuration

Create docker-compose.yml in the project root:

```
services:
  frontend:
    build: ./frontend
    ports:
      - "3000:80"
    depends_on:
      - backend
  backend:
    build: ./backend
    ports:
      - "5000:5000"
    environment:
      PORT: 5000
      MONGO_URI: mongodb://mongo:27017/campusconnect
      REDIS_URL: redis://redis:6379
      JWT_SECRET: ${JWT_SECRET}
      FRONTEND_URL: ${FRONTEND_URL}
    depends_on:
      - mongo
      - redis
  mongo:
    image: mongo:8
    volumes:
      - mongo_data:/data/db
  redis:
    image: redis:7-alpine
volumes:
  mongo_data:
```

Create a root .env file containing development configuration:

```
JWT_SECRET=replace_with_a_long_random_secret
FRONTEND_URL=http://localhost:3000
```

Do not commit .env to Git. Use a secret-management solution for production.

Important configuration notes:

- Add an nginx.conf file with SPA fallback routing if React Router is used.
- Configure the frontend's API URL to reach the backend through the browser-accessible address, not the internal Docker hostname backend.
- Make the backend wait for MongoDB and Redis to become healthy, preferably using Compose health checks.
- Add a MongoDB initialization strategy if database indexes or seed users are required.

Start the application from the project root:
docker compose up –build

Check the running services:

docker compose ps
docker compose logs backend

Open the frontend at http://localhost:3000 and the backend at http://localhost:5000.

Expected Output

The completed CampusConnect application should demonstrate the following:

1. Users can register and log in securely.
2. Admins can create, update, and delete events and announcements.
3. Students can view events, search, and RSVP.
4. Students receive live announcement notifications.
5. Redis caches event responses for 60 seconds and invalidates stale data after event changes.
6. Unauthorized users receive appropriate 401 or 403 responses.
7. Automated backend and frontend tests execute successfully.
8. The frontend, backend, MongoDB, and Redis run together through Docker Compose.

Result

The CampusConnect application is designed to provide a secure, role-based college event management platform using the MERN stack. JWT authentication and RBAC protect resources, Socket.io delivers real-time notifications, Redis improves repeated-read performance, and Docker Compose simplifies deployment. Automated testing and security middleware help improve application reliability.

Conclusion:

This lab demonstrates the practical integration of secure REST APIs, real-time communication, caching, frontend state management, automated testing, and containerized deployment in a full-stack web application.